

CS339: Abstractions and Paradigms for Programming

ADTs and Type Classes (Cont.)

Manas Thakur
CSE, IIT Bombay



Autumn 2025

Recap

- We learnt how could we create new *algebraic data types* in Haskell:

```
data Shape = Circle Float | Rectangle Float Float
```

- Pattern matching helps define operations:

```
area :: Shape -> Float  
area (Circle r) = pi * r * r  
area (Rectangle ...
```

- We also learnt about *type parameters* that allow us to supply any type later:

```
data Maybe a = Nothing | Just a
```

- Question: Can we create recursive types?



Recursive Types

- What does this type describe?

```
data Nat = Zero | Succ Nat
```

- Does this remind you of something from *pre-Quiz1* time?

- Church numerals!

```
nat2int :: Nat -> Int
nat2int Zero      = 0
nat2int (Succ n) = 1 + nat2int n
```

```
add :: Nat -> Nat -> Nat
add m n = int2nat (nat2int m + nat2int n)
```

↗
Classwork



Type Classes

- What's the problem with the following types for `+`:

Too specific $\begin{cases} (+) :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \\ (+) :: \text{Float} \rightarrow \text{Float} \rightarrow \text{Float} \end{cases}$
Too generic $\rightarrow (+) :: a \rightarrow a \rightarrow a$

- We would like to define `(+)` for only *number-like* types.
- Check the type of Haskell's built-in sort function:

```
> import Data.List
> :t sort
> sort :: Ord a => [a] -> [a]
```

- It sorts lists containing elements of any type `a` that belongs to the **type class** `Ord` ("`Ord a`" being a **type constraint**).



Type Classes (Cont.)

- Check the type of + now:

```
> :t (+)  
> (+) :: Num a => a -> a -> a
```

a is a type variable

- Here is the declaration of the type class Num from Haskell:

```
class (Eq a, Show a) => Num a where  
  (+), (-), (*) :: a -> a -> a  
  negate :: a -> a  
  abs, signum :: a -> a  
  fromInteger :: Integer -> a
```

- Class Num is a subclass of type classes Eq and Show.
- Any subclass of Num would need to define (+), (-), negate, etc.



Instantiating Type Classes

- A type can be made an instance of a type class by *instantiating* that class:

```
class Eq a where
  (==), (/=) :: a -> a -> Bool
  x /= y = not (x == y)
```

- Now any type can instantiate Eq by defining == (/= is already defined):

```
data Bool = False | True

instance Eq Bool where
  False == False = True
  True   == True   = True
  _      == _      = False
```



Deriving from Type Classes

```
data Student = Student { name :: String,  
                          rollNum :: Int }
```

- We can create *named* values as follows:

```
stud = Student { name = "harry", rollNum = 123 }
```

- But we can't "show" stud yet! (Notice the error.)
- Derive from something that knows how to *show*:

```
data Student = Student { name :: String,  
                          rollNum :: Int  
                        } deriving Show
```

- Now typing stud on the prompt works!



Deriving from Type Classes

- Could we have done the following instead?

```
data Student = Student { name :: String,  
                        rollNum :: Int }  
instance Show Student
```

- The “deriving” version generates a definition; the “instance” one does not.
- The following would work:

```
data Student = Student { name :: String,  
                        rollNum :: Int }  
instance Show Student where  
  show (Student n r) = "Student { name = " ++ show n ++  
                        ", rollNum = " ++ show r ++ " }"
```



The Expression Problem

➤ ADTs:

The Functional Style

- Adding new operations (functions) is easy, particularly with pattern matching
- Adding new types is difficult (have to modify the top-level data declaration — breaks abstraction barrier)

➤ Typeclasses:

The OO Style

- Adding new types (classes) is easy — just create a new instance
- Adding new operations is difficult (have to modify the top-level typeclass — breaks abstraction barrier)
- Typeclasses are like Java interfaces (Haskell has everything!)

